Introduction to RAG (Retrieval-Augmented Generation)

A Beginner-Friendly Lesson for Students Starting Their AI Journey

Target Audience: Beginners with no prior AI knowledge
Level: Beginner
Estimated Reading Time: 10–15 minutes

---

1. Learning Objectives

By the end of this lesson, you will be able to:

- Understand what RAG means.
- Explain why RAG is useful.
- Understand how RAG works step by step.
- Identify the main components of a RAG system.
- Understand the difference between RAG and a normal AI chatbot.
- Recognize real-world applications of RAG.

2. What is RAG?

RAG stands for **Retrieval-Augmented Generation**.

Let's understand these three words:

- **Retrieval:** Finding useful information from stored documents or other sources.
- **Augmented:** Adding extra information to improve an answer.
- **Generation:** Creating a new answer using an AI model.

In simple words, RAG is a technique that helps an AI system find relevant information from external sources and use that information to generate a better answer.

Imagine you are preparing for an examination. Instead of answering every question from memory, you open your textbook, find the relevant topic, read the information and then write your answer.

RAG works in a similar way.

It first searches for useful information and then gives that information to an AI model so it can prepare an answer.

Simple definition:

RAG is a technique that combines information retrieval with AI-generated answers.

3. Why Do We Need RAG?

Before understanding RAG, let's understand a problem with normal AI chatbots.

Imagine you ask an AI chatbot:

"What is my college's attendance policy?"

The chatbot may not know your college's latest attendance policy because that information might not be included in its training data.

It might give an incorrect answer or make up information.

This is where RAG becomes useful.

Instead of depending only on what the AI model learned during training, RAG allows the system to search relevant documents and use their information to answer the question.

Problems RAG Helps Address

1. **Limited knowledge:** An AI model may not know about private or organization-specific information.
2. **Outdated information:** Its training data may not contain the latest updates.
3. **Incorrect answers:** AI models can sometimes produce information that sounds correct but is not supported by facts.
4. **Private documents:** Organizations may want AI systems to answer questions using their own documents.

RAG helps address these problems by providing relevant external information to the AI model.

4. How Does RAG Work?

Let's understand the working process through a simple example.

Imagine a college has uploaded a document containing its attendance rules.

A student asks:

"What is the minimum attendance required in my college?"

A RAG system follows these steps.

Step 1: Prepare the Documents

First, the system collects information from sources such as:

- PDF files
- Word documents
- Text files
- Websites
- Company knowledge bases

For example, the college attendance policy is uploaded into the system.

Step 2: Split Documents into Smaller Parts

Large documents are divided into smaller sections called **chunks**.

Why?

Because searching through smaller sections is easier than searching through one very large document.

Example:

Original document:

"Students must maintain at least 75% attendance in every subject. Students below this requirement may not be allowed to appear for examinations."

This information can be divided into smaller chunks while preserving its meaning.

Step 3: Convert Text into Embeddings

An embedding is a numerical representation of text.

In simple words, the system converts text into numbers so that a computer can compare the meaning of different pieces of text.

For example:

- "Minimum attendance requirement"
- "How many classes must I attend?"

These sentences use different words but have similar meanings.

Embeddings help the system identify that they are related.

Step 4: Store Information in a Vector Database

The embeddings are stored in a special database called a **Vector Database**.

A vector database stores numerical representations of information and helps find information with similar meaning.

Examples include:

- FAISS
- Chroma
- Pinecone

Think of it as a smart digital library where information can be found based on meaning, not only exact words.

Step 5: User Asks a Question

Now the student asks:

"What is the minimum attendance required?"

The system converts the question into an embedding using the same embedding model or a compatible model.

Step 6: Retrieve Relevant Information

The system searches the vector database and finds the chunks that are most relevant to the student's question.

For example, it retrieves:

"Students must maintain at least 75% attendance in every subject."

This process is called **Retrieval**.

Step 7: Generate the Answer

The retrieved information is provided to the Large Language Model (LLM), along with the student's original question.

The LLM uses this information to generate a natural-language answer.

Example:

"According to your college attendance policy, you must maintain at least 75% attendance in every subject."

This process is called **Generation**.

Step 8: Display the Final Answer

The student receives the answer through the chatbot.

The complete process is:

Documents → Chunks → Embeddings → Vector Database → Question → Retrieval → LLM → Answer

5. Understanding the Main Components of RAG

Component	Simple Explanation
Documents	Information sources such as PDFs and text files
Chunking	Dividing large documents into smaller sections
Embeddings	Converting text into numerical representations
Embedding Model	A model that creates embeddings
Vector Database	Stores embeddings and helps find similar information
Retriever	Finds relevant information for a question
LLM	Generates the final answer using the question and retrieved information

What is an LLM?

LLM stands for Large Language Model.

It is an AI model trained on large amounts of text to understand and generate human language.

Examples include GPT and Gemini.

An LLM can answer questions, summarize information and generate text.

In RAG, the LLM uses the retrieved information to prepare an answer.

6. RAG Architecture

A RAG system generally has two main stages.

A. Indexing Stage

This stage prepares information before users ask questions.

1. Collect documents.
2. Divide documents into chunks.
3. Convert chunks into embeddings.
4. Store embeddings in a vector database.

B. Retrieval and Generation Stage

This stage happens when a user asks a question.

1. Receive the user's question.

2. Convert the question into an embedding.
3. Search for relevant chunks.
4. Retrieve the matching information.
5. Provide the retrieved information to the LLM.
6. Generate and display the answer.

Remember:

Indexing prepares the knowledge. Retrieval and generation use that knowledge to answer questions.

7. RAG vs. Normal LLM

Normal LLM	RAG-based System
Answers mainly using knowledge learned during training and the current conversation	Uses retrieved external information as additional context
May not know private company documents	Can search connected company documents
May provide outdated information	Can use updated documents if the knowledge source is maintained
May generate unsupported answers	Can ground answers in retrieved information
Does not require a retrieval system	Requires a retrieval process and a knowledge source

RAG does not guarantee that every answer will be correct. Its quality depends on the documents, retrieval process and generated response.

8. Real-World Applications of RAG

RAG is useful in many industries.

1. Education

Students can ask questions about uploaded textbooks, notes and study materials.

Example: "Explain Chapter 3 from my uploaded textbook."

2. Healthcare

Healthcare professionals can search information from approved medical documents and guidelines.

3. Customer Support

Companies can build chatbots that answer questions using product manuals, FAQs and company policies.

4. Banking

Employees can search internal banking policies and documentation.

5. Software Development

Developers can ask questions about project documentation, technical guides and code repositories.

9. Advantages of RAG

- Provides answers using external information.
- Can work with private and organization-specific documents.
- Makes it easier to update the knowledge source without retraining the LLM.
- Can provide answers grounded in retrieved documents.
- Can support references or citations to source documents.

10. Limitations of RAG

RAG is useful, but it also has limitations.

1. **Poor document quality:** Incorrect information in the documents can lead to incorrect answers.
2. **Retrieval errors:** The system may retrieve irrelevant information.
3. **Missing information:** If the required information is not available, the system may not answer correctly.
4. **Processing time:** Searching documents and generating answers takes additional time.
5. **Cost:** Using embedding models, databases and LLM APIs can involve costs.

Therefore, a good RAG system needs proper document preparation, retrieval and answer evaluation.

11. Simple Example: How RAG Answers a Question

Let's revise everything with one complete example.

Situation: A company uploads its employee leave policy into a RAG system.

Employee's question:

"How many paid leaves can I take in a year?"

What happens behind the scenes?

1. The system receives the employee's question.
2. It searches the company's leave policy.
3. It finds the relevant section.
4. It sends the information to the LLM.
5. The LLM generates an answer using the retrieved policy.

Example retrieved information:

"Employees are entitled to 20 paid leaves per calendar year."

Generated answer:

"According to the company's leave policy, employees are entitled to 20 paid leaves per calendar year."

The answer is based on the information retrieved from the company's document.

Note: The leave policy above is fictional and used only for learning.

12. Key Takeaways

Let's remember the most important points:

- RAG stands for Retrieval-Augmented Generation.
- It combines information retrieval with AI-generated answers.
- It allows an AI system to use external information.
- Documents are divided into smaller chunks.
- Embeddings represent text as numbers.
- Vector databases help retrieve information based on similarity.
- An LLM uses retrieved information to generate an answer.
- RAG can reduce unsupported answers, but it does not guarantee accuracy.

13. Test Your Understanding

Try answering these questions without looking at the lesson.

Q1. What is the full form of RAG?

A. Random Answer Generation
B. Retrieval-Augmented Generation
C. Retrieval Automatic Gateway
D. Real-time Answer Generator

Correct Answer: B

Q2. Why are documents divided into chunks?

A. To delete unnecessary information
B. To make searching and retrieving information easier
C. To train the LLM again
D. To convert documents into images

Correct Answer: B

Q3. What is an embedding?

A. A numerical representation of text
B. A type of PDF
C. A programming language
D. A chatbot

Correct Answer: A

Q4. What is the purpose of a vector database?

A. To create websites
B. To store and search numerical representations of information
C. To design images
D. To write programming code

Correct Answer: B

Q5. Does RAG retrain the LLM every time a user asks a question?

A. Yes
B. No

Correct Answer: B

Q6. Which component generates the final natural-language answer?

A. Vector Database
B. Document Loader
C. LLM
D. Chunking Process

Correct Answer: C

---

14. Final Conclusion

Retrieval-Augmented Generation (RAG) is an important technique in modern Generative AI.

It helps AI systems answer questions using relevant external information instead of depending only on their previously learned knowledge.

By combining document retrieval, embeddings, vector databases and Large Language Models, RAG makes it possible to build useful AI applications for education, healthcare, business and many other fields.

In one sentence: RAG helps AI find the right information before generating an answer.